

森林氧吧·AI智慧康养系统 —— 概要设计说明书

项目名称	森林氧吧·AI智慧康养系统
文档版本	V1.2
编制日期	2026-09-15
对应需求	《森林氧吧·AI智慧康养系统 —— 需求分析文档》V1.1
文档性质	概要设计（High-Level Design）
密级	秘密
文档编号	D0000-PPC-FOAI-PPD-2026

1 文档概述

1.1 文档目的和范围

本文档在《森林氧吧·AI智慧康养系统 —— 需求分析文档》V1.1 的基础上，给出系统的总体设计方案，明确体系结构、技术选型、模块划分、接口定义、数据结构、核心业务流程、出错处理与安全保密设计，作为详细设计、编码实现与测试验收的共同依据。

文档范围：覆盖系统总体架构、运行部署、模块划分、数据库设计、接口设计、核心流程、安全与可维护性设计。不包含详细到函数级实现与具体代码清单（该部分见详细设计文档与源码）。

预期读者：项目开发人员、测试人员、指导教师与验收人员。

1.2 术语/缩略语

序号	术语/缩略语	说明
1	API	应用程序编程接口
2	服务日期 <code>serviceDate</code>	预约对应的绝对日历日期，格式 <code>YYYY-MM-DD</code> ，落库存储
3	时段 <code>slot</code>	一天内固定四个服务时段，取值 0~3
4	预约窗口	可预约的日期范围，默认「今/明/后天」共 3 天
5	过账 Rollover	服务日期已过的 <code>booked</code> 预约自动流转为 <code>done</code>
6	空档	某日期某时段无预约记录，排班视图显示为空闲
7	SPA	单页应用（Single Page Application）
8	JWT	JSON Web Token，无状态身份令牌
9	仓储 <code>repo</code>	数据存储层，业务服务层依赖的统一数据访问接口
10	DTO	数据传输对象，接口返回的字段集合
11	ESM	ECMAScript Modules，Node.js 模块规范
12	CORS	跨域资源共享

1.3 参考文档

序号	文档名	作者	时间	版本
1	《森林氧吧·AI智慧康养系统 —— 需求分析文档》	项目组	2026-09	V1.1
2	后端源码 <code>server/</code> 、前端源码 <code>src/</code>	项目组	2026-09	—
3	数据库建表脚本 <code>server/sql/schema-mysql.sql</code>	项目组	2026-09	—
4	演示数据脚本 <code>server/sql/seed.sql</code>	项目组	2026-09	—

2 系统结构图

系统采用四层结构，前端与后端通过 HTTP/JSON 交互，存储层可插拔（MySQL / JSON 文件）。

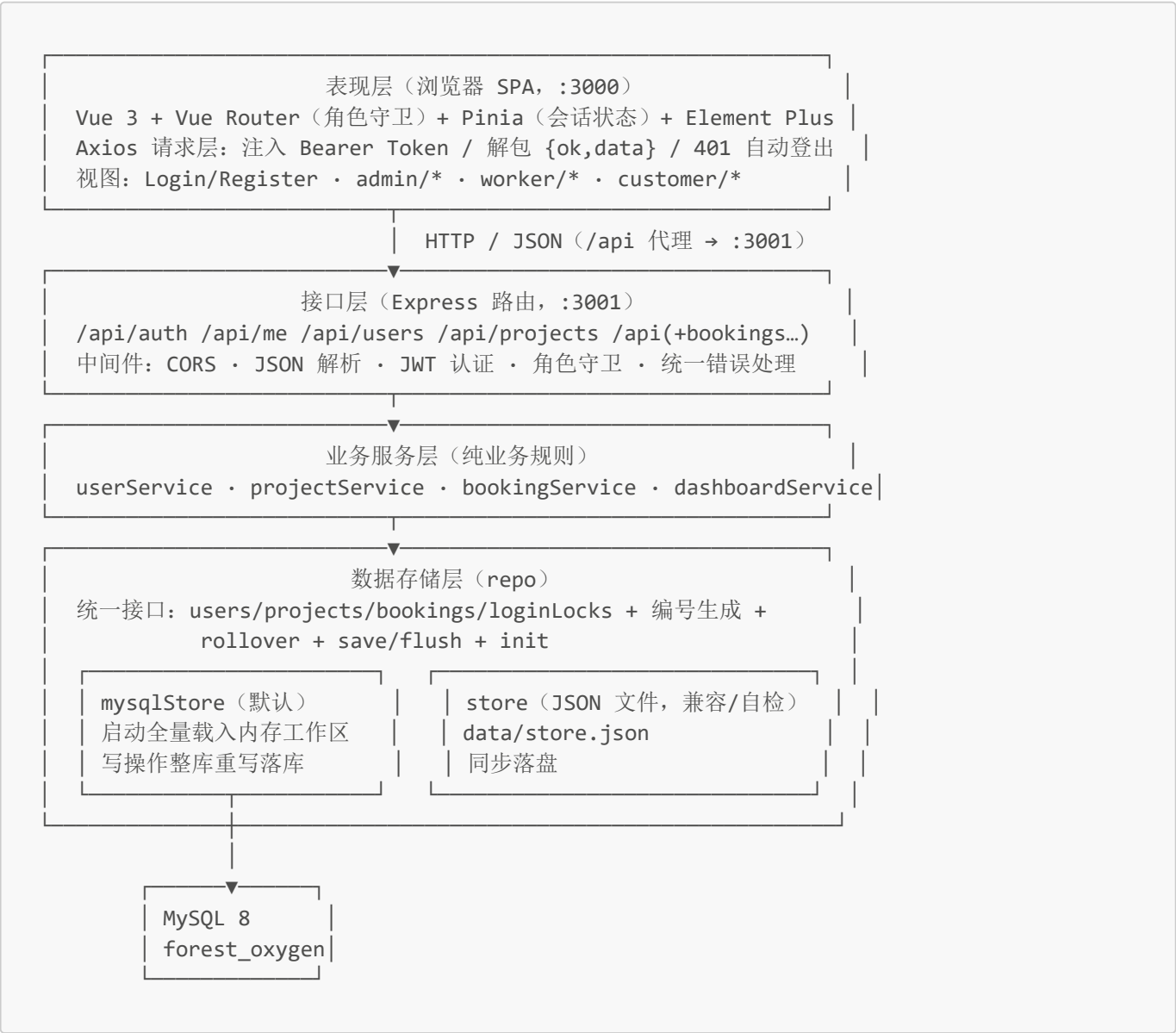


图 2-1 系统结构图

架构要点：

- 1. 存储层可替换：业务服务层只依赖 `repo` 暴露的「内存集合 + 编号生成 + 持久化」约定，切换 `STORE=mysql|file` 时服务层与前端零改动。

2. **MySQL 驱动的内存工作区模型**：启动时把整库一次性载入内存作为工作区，读与规则判断保持同步；每次写操作触发一次串行「整库重写」异步落库，写接口在响应前 `await repo.flush()` 等待落库完成。
3. **无状态鉴权**：服务端不保存会话，JWT 承载 `{id, role, phone}`，前端存于 `localStorage` 并随请求头携带。

3 模块详细概述

系统按角色划分为三大功能域，前端与后端模块划分如下。模块总体说明：前端负责表现与交互，后端负责业务规则与数据持久化，二者通过 REST API 解耦。

3.1 前端模块

3.1.1 前端模块功能定义

序号	功能点	功能点详细说明
1	请求层	<code>src/api/http.js</code> ：baseURL、Bearer 注入、响应解包、401 清会话回登录、 <code>ApiError</code> 归一
2	接口封装	<code>src/api/index.js</code> ：与后端路由一一对应的 <code>api.*</code> 方法
3	会话状态	<code>src/stores/auth.js</code> ：token/id/role/user，登录、注册、拉取/更新本人资料、登出
4	路由	<code>src/router/index.js</code> 、 <code>menus.js</code> ：路由表、按角色守卫、各角色导航菜单与默认落地页
5	布局	<code>src/layout/RoleLayout.vue</code> ：顶栏 + 园区地图/环境 + 角色导航 + 会话超时
6	通用组件	<code>src/components/</code> ：预约对话框 <code>BookingDialog</code> 、图表封装 <code>BaseChart</code> 、看板卡片 <code>StatCards/EnvBoard</code>
7	视图	<code>src/views/</code> ：admin/、worker/、customer/、common/、Login、Register
8	工具	<code>src/utils/config.js</code> 、 <code>dates.js</code> 、 <code>echarts.js</code> ：时段/角色/环境阈值常量、日期映射、ECharts 按需注册

3.1.2 前端模块结构

```
src/
├── api/
│   ├── http.js      # Axios 实例、拦截器
│   └── index.js      # api.* 接口封装
├── stores/
│   └── auth.js       # Pinia 会话状态
├── router/
│   ├── index.js      # 路由表 + 角色守卫
│   └── menus.js       # 角色导航菜单
├── layout/
│   └── RoleLayout.vue # 统一布局
├── components/
│   ├── BookingDialog.vue
│   ├── BaseChart.vue
│   ├── StatCards.vue
│   └── EnvBoard.vue
└── views/
```



图 3-1 前端模块结构图

3.1.3 前端模块类图

前端以组合式 API + Pinia 为核心，主要类/模块关系如下：

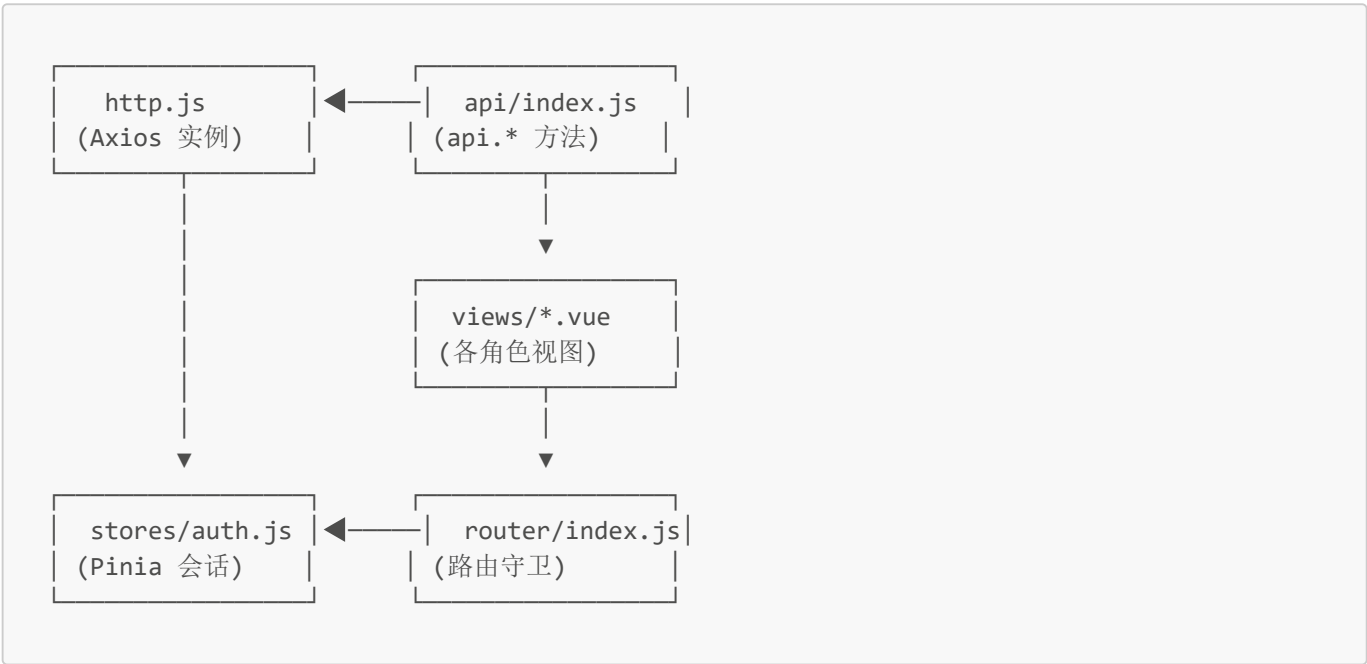


图 3-2 前端模块类图

表 3-1 前端模块接口说明

序号	模块名称	接口函数	函数说明
1	http.js	request(config)	统一请求，注入 Token、解包响应
2	http.js	ApiError	错误归一化类
3	auth.js	login(phone, password)	登录并写入会话
4	auth.js	register(payload)	注册
5	auth.js	fetchMe()	拉取本人资料
6	auth.js	updateMe(payload)	更新本人资料/改密
7	auth.js	logout()	登出并清会话
8	router/index.js	beforeEach	角色守卫

3.2 后端模块

3.2.1 后端模块功能定义

序号	功能点	功能点详细说明
1	入口	src/index.js：依 STORE 装配仓储 → init() → 启动 HTTP 服务；优雅退出等待落库
2	应用组装	src/app.js：挂载 CORS、JSON 解析、健康检查、五组路由、404 与错误处理
3	配置	src/config.js：解析 .env，优先级 进程环境变量 > .env > 默认值
4	路由层	src/routes/：auth 登录注册、me 本人、users 用户管理(admin)、projects 项目、bookings 预约/排班/看板
5	中间件	src/middleware/middleware.js：authRequired、requireRole、ah 异步包装、windowDays 校验、404、统一错误
6	服务层	src/services/：userService、projectService、bookingService、dashboardService
7	存储层	src/store/：mysqlStore.js、store.js、catalog.js（8 个项目目录清单）
8	工具	src/utils/：errors 异常与响应、validate 字段校验、datetime 日期工具、token JWT
9	脚本	scripts/：selfcheck.js（72 项断言）、gen-mysql-seed.mjs（灌数生成器）

3.2.2 后端模块结构

```
server/
├── src/
│   ├── index.js      # 入口：装配仓储、启动服务
│   ├── app.js        # Express 应用组装
│   ├── config.js     # 配置解析
│   ├── routes/       # 路由层
│   │   ├── auth.js
│   │   ├── me.js
│   │   ├── users.js
│   │   ├── projects.js
│   │   └── bookings.js
│   ├── middleware/
│   │   └── middleware.js
│   ├── services/     # 业务服务层
│   │   ├── userService.js
│   │   ├── projectService.js
│   │   ├── bookingService.js
│   │   └── dashboardService.js
│   ├── store/        # 存储层
│   │   ├── mysqlStore.js
│   │   ├── store.js
│   │   └── catalog.js
│   └── utils/
│       ├── errors.js
│       ├── validate.js
│       ├── datetime.js
│       └── token.js
```

```
├── sql/
│   ├── schema-mysql.sql
│   └── seed.sql
└── scripts/
    ├── selfcheck.js
    └── gen-mysql-seed.mjs
```

图 3-3 后端模块结构图

3.2.3 后端模块类图

后端以「路由 → 服务 → 存储」三层单向依赖为核心，主要模块关系如下：



图 3-4 后端模块类图

表 3-2 后端模块接口说明

序号	模块名称	接口函数	函数说明
1	userService	login(phone, password)	登录校验、失败锁定、签发 JWT
2	userService	register(payload)	注册护工/客户
3	userService	list(role)	用户列表
4	userService	create(payload)	管理员建号
5	userService	update(id, payload)	编辑用户
6	userService	schedule(id, days)	护工排班（裁剪隐私字段）
7	projectService	list(role)	项目列表
8	projectService	occupancy(id, date, days)	时段占用/余量
9	projectService	create(payload)	新增项目
10	projectService	update(id, payload)	编辑/启停项目
11	bookingService	book(customerId, payload)	预约核心校验链
12	bookingService	cancel(customerId, id)	取消预约

序号	模块名称	接口函数	函数说明
13	bookingService	myWindow(customerId)	客户未来窗口排布
14	bookingService	myHistory(customerId)	客户历史
15	bookingService	workerSchedule(workerId, days)	护工本人排班
16	bookingService	rollover()	过期预约过账
17	dashboardService	overview(user)	角色看板统计

3.3 模块调用关系（典型请求时序）

以「客户提交预约」为例：



图 3-5 客户提交预约时序图

4 数据库设计

4.1 数据库引擎概述

项目	说明
数据库	MySQL 8.x
存储引擎	InnoDB（支持事务、行级锁、外键）
字符集	utf8mb4，排序规则 utf8mb4_general_ci
库名	forest_oxygen
连接驱动	mysql2（Promise API、预编译参数、dateStrings 保持日期语义）
设计原则	数据不物理删除，预约状态机流转；绝对日期存储；索引直接支撑冲突与容量查询

4.2 数据库概要设计

4.2.1 E-R 关系图

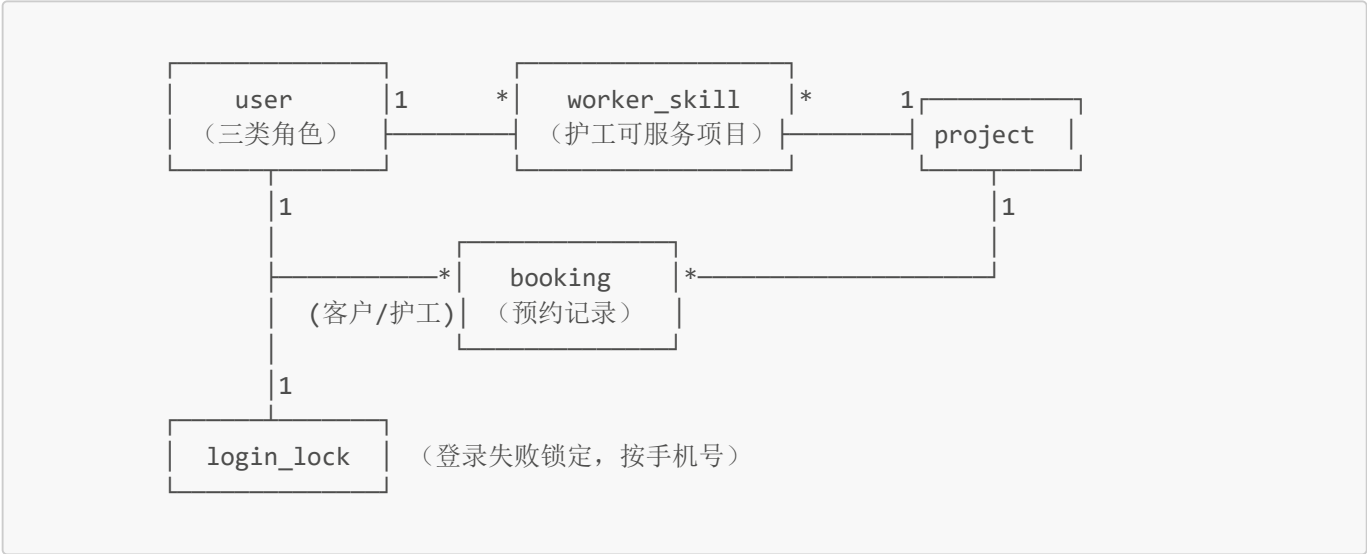


图 4-1 数据库 E-R 关系图

关系说明：**user** 同时承载管理员/护工/客户；护工与项目为多对多 (**worker_skill**)；**booking** 关联客户、护工、项目各一方；登录锁定表独立按手机号维护。

4.2.2 表结构设计

(1) user（用户，三类角色共用）

字段	类型	约束	说明
id	CHAR(6)	PK	6 位编号，首位 0 管理员 / 1 护工 / 2 客户
role	ENUM	NOT NULL	admin / worker / customer
phone	VARCHAR(11)	UNIQUE	登录账号
password_hash	VARCHAR(100)	NOT NULL	bcrypt 哈希
name	VARCHAR(30)	NOT NULL	姓名
age	TINYINT UNSIGNED	DEFAULT 0	0-150
gender	VARCHAR(4)	DEFAULT '保密'	男/女/保密
salary	DECIMAL(10,2)	DEFAULT 0	护工月薪
status	ENUM	DEFAULT 'active'	护工在职 active / 离职 resigned
allergy / disease / preference	VARCHAR	DEFAULT ''	客户健康档案（忌口/疾病史/偏好）
created_at	DATETIME	DEFAULT NOW	创建时间

索引：**uk_user_phone(phone)**、**idx_user_role(role)**。

(2) worker_skill（护工可服务项目，多对多）

字段	类型	约束
worker_id	CHAR(6)	PK, FK→user(id)
project_id	VARCHAR(8)	PK

(3) project（服务项目）

字段	类型	说明
id	VARCHAR(8)	PK，如 P001（自动生成）
name	VARCHAR(50)	项目名称
location	VARCHAR(60)	地点
fee	DECIMAL(10,2)	费用（元/次）
capacity	SMALLINT UNSIGNED	同一时段最大容量
status	ENUM	active 进行 / disabled 停用
duration	VARCHAR(20)	单次时长
flow	VARCHAR(500)	服务流程
suitable	VARCHAR(200)	适合人群
taboo	VARCHAR(300)	禁忌事项
created_at	DATETIME	创建时间

索引：idx_project_status(status)。

(4) booking（预约记录，含历史，状态流转不物理删除）

字段	类型	说明
id	VARCHAR(24)	PK，如 B00001
customer_id / worker_id / project_id	CHAR(6)/CHAR(6)/VARCHAR(8)	关联三方
service_date	DATE	绝对服务日期 YYYY-MM-DD
slot	TINYINT	时段 0-3
status	ENUM	booked → done / cancelled
created_at / updated_at	DATETIME	时间戳
cancelled_at	DATETIME NULL	取消时间

索引：idx_booking_date_slot(service_date,slot)、
idx_booking_project(project_id,service_date,slot)、idx_booking_customer(customer_id,status)、
idx_booking_worker(worker_id,service_date,slot) —— 直接支撑「客户/项目/护工在指定日期+时段的冲突与容量查询」。

(5) login_lock（登录失败锁定）

字段	类型	说明
phone	VARCHAR(11)	PK
fail_count	INT UNSIGNED	连续失败次数
locked_until	DATETIME NULL	锁定截止时间

4.2.3 内存工作区与持久化策略

- **载入**：`mysqlStore.init()` 启动时将 5 张表全量读入内存（`worker_skill` 合并为 `user.skills` 数组）。
- **写入**：业务写操作调用 `repo.save()`，把「整库重写」压入一条 **串行 Promise 队列**（`DELETE` 全表 → 批量 `INSERT`，`SET FOREIGN_KEY_CHECKS=0` 包裹，事务提交）；写接口在响应前 `await repo.flush()` 等待本次落库，保证「接口成功 = 数据已落库」。
- **容错**：落库失败记录日志、不中断进程，内存仍为准，后续任一写操作会再次整库重写**自愈**；MySQL 连接断开时自动重连。
- **可选 JSON 存储**：`STORE=file` 时使用 `store.js`，数据存 `server/data/store.json`，采用「临时文件 + `rename`」原子写入，用于旧数据兼容与 `selfcheck` 自检引擎。

4.2.4 编号生成规则

实体	规则	示例
用户	角色前缀（admin 0 / worker 1 / customer 2）+ 角色内最大序号+1，补足 5 位	200010
项目	P + 最大序号+1，补足 3 位	P009
预约	B + 最大序号+1，补足 5 位	B00048

编号从当前内存集合推导最大值生成，重启后不重复。

5 接口设计

5.1 全局变量

系统无进程级可变全局变量。所有配置经 `server/.env` 注入，由 `src/config.js` 解析为只读配置对象；运行时状态（内存工作区、落库队列）封装在 `repo` 内部，不对外暴露。关键可配置项如下：

表 5-1 关键可配置项（`server/.env`）

变量	含义	默认
PORT	后端端口	3001
STORE	存储引擎 <code>mysql</code> / <code>file</code>	<code>file</code> （ <code>.env</code> 中设为 <code>mysql</code> ）
DB_HOST / DB_PORT / DB_USER / DB_PASSWORD / DB_NAME	MySQL 连接	127.0.0.1 / 3306 / root / - / forest_oxygen
BOOKING_WINDOW_DAYS	可约窗口天数	3
LOGIN_MAX_FAILS / LOGIN_LOCK_MINUTES	失败锁定阈值/时长	5 / 15

变量	含义	默认
BCRYPT_ROUNDS	哈希强度	10
JWT_SECRET / JWT_EXPIRES_MS	令牌密钥/有效期	开发默认 / 8 小时
ADMIN_PHONE / ADMIN_PASSWORD / ADMIN_NAME	空库引导管理员	138000000001 / 12345678 / 系统管理员
DATA_FILE	STORE=file 数据 路径	./data/store.json
ENV_REFRESH_MS	环境数据模拟刷新 间隔	6000

前端会话存储：

键	内容	用途
fo_token	JWT 字符串	请求头 Authorization
fo_auth	{id, role} JSON	刷新页面后路由守卫即时判断角色，无需等待网络

5.2 模块间接口函数设计

5.2.1 接口约定

- **风格**：RESTful 风格，基于 HTTP/JSON，路径统一前缀 /api。
- **鉴权**：除 /api/health、/api/auth/login、/api/auth/register 外，均需请求头 Authorization: Bearer <JWT>。
- **成功响应**：{ "ok": true, "data": <对象>, "msg"?: <提示> }；纯提示型接口可只含 msg。
- **失败响应**：{ "ok": false, "code": <业务码>, "msg": <人类可读信息> }，HTTP 状态码语义化（400/401/403/404/409/429/500）。
- **前端解包**：请求层自动返回 data 部分；失败抛 ApiError{status, code, msg}。

5.2.2 REST API 清单

表 5-2 REST API 清单

#	方法	路径	权限	功能	主要入参 / 返回
1	GET	/api/health	公开	健康检查	返回服务名/版本/时间
2	POST	/api/auth/login	公开	登录	{phone,password} → {token,user}
3	POST	/api/auth/register	公开	注册 (护工/客户)	{role,phone,password,name,gender,age,...} → {id,user}

#	方法	路径	权限	功能	主要入参 / 返回
4	GET	/api/me	登录	当前用户资料	→ user (不含密码哈希)
5	PUT	/api/me	登录	更新本人资料/改密	客户可改健康档案；护工只读仅可改密； {oldPassword,newPassword}
6	GET	/api/users?role=	管理员	用户列表	role=worker customer → {total,items}
7	POST	/api/users	管理员	新建账号	密码缺省 12345678 → {id,user}
8	GET	/api/users/:id	登录	用户详情	→ user
9	PUT	/api/users/:id	管理员	编辑用户	白名单字段、护工状态/薪资/技能、重置密码
10	GET	/api/users/:id/schedule?days=	管理员	护工排班	→ {worker,days,base,cells} (不含健康档案)
11	GET	/api/users/:id/bookings?mode=	管理员	客户预约/历史	mode=window history → {customer,mode,items}
12	GET	/api/projects	登录	项目列表	管理员含停用，客户仅进行中 → {total,items}
13	GET	/api/projects/:id	登录	项目详情	→ project
14	GET	/api/projects/:id/occupancy?date=&days=	登录	时段占用/余量	→ {project,days:[{serviceDate,slots:[{slot,count,capacity,left,available}]}}}
15	POST	/api/projects	管理员	新增项目	自动生成 Pxxx 编号 → {id,project}

#	方法	路径	权限	功能	主要入参 / 返回
16	PUT	/api/projects/:id	管理员	编辑/启停项目	全字段覆盖式更新
17	GET	/api/bookings/my	客户	未来窗口排布 (含空档)	→ {days,base,rows}
18	GET	/api/bookings/my/history	客户	历史 (已完成/已取消)	→ {items} (日期倒序)
19	POST	/api/bookings	客户	提交预约	{projectId,serviceDate,slot} → 预约单 (含匹配护工)
20	POST	/api/bookings/:id/cancel	客户	取消预约	仅本人、仅 booked 态
21	GET	/api/schedule/me?days=	护工	本人排班	→ {days,base,cells} (本人视角含客户健康档案)
22	GET	/api/schedule/me/history	护工	已完成服务回顾	→ {items}
23	GET	/api/dashboard	管理员/客户	角色看板统计	管理员：计数 + 近期待服务预约；客户：个人概览

5.2.3 业务错误码

表 5-3 业务错误码

错误码	HTTP	含义
BAD_BODY / BAD_PHONE / BAD_PASSWORD / BAD_AGE / BAD_GENDER / BAD_NAME / BAD_ROLE / BAD_SLOT / BAD_SKILLS / BAD_STATUS / BAD_FEE / BAD_CAPACITY / BAD_LOCATION / BAD_DATE / BAD_DAYS	400	入参校验失败 (逐字段)
OUT_OF_WINDOW	400	预约日期超出今/明/后天窗口

错误码	HTTP	含义
PROJECT_DISABLED	400	项目已停用，不可预约
OLD_PASSWORD_WRONG	400	原密码不正确
ALREADY_DONE / ALREADY_CANCELLED	400	预约已完成后/已取消，无法再取消
UNAUTHORIZED	401	未登录或 token 失效
PHONE_NOT_REGISTERED	401	手机号未注册
WRONG_PASSWORD	401	密码错误（含剩余次数提示）
WORKER_RESIGNED	401	护工已离职，禁止登录
FORBIDDEN	403	角色无权访问
NOT_FOUND	404	资源不存在 / 接口不存在
PHONE_TAKEN	409	手机号已被注册
SLOT_TAKEN	409	客户该日期时段已有预约
CAPACITY_FULL	409	该时段预约人数已满
NO_WORKER	409	该时段无匹配的空闲护工
ACCOUNT_LOCKED	429	失败次数过多，账号锁定
INTERNAL	500	服务器内部错误

5.2.4 关键接口函数详细信息

表 5-4 bookingService.book() 接口说明

项	内容
函数名	book
文件名	server/src/services/bookingService.js
功能概要	客户提交预约，完成校验链后生成 booked 记录并落库
参数	customerId: string (l, 客户编号); payload: {projectId, serviceDate, slot} (l, 预约请求体)
返回值	booking (预约单对象，含匹配护工信息)

项	内容
详细说明	依次执行 rollover → 日期窗口校验 → 项目可约校验 → 客户时段去重 → 容量校验 → 候选护工筛选 → 随机分配 → 生成记录 → <code>repo.save()</code>
使用注意事项	纯同步函数；任一校验失败抛 <code>HttpError</code> ，不产生部分写入；调用方需 <code>await repo.flush()</code> 确保落库

表 5-5 `bookingService.cancel()` 接口说明

项	内容
函数名	<code>cancel</code>
文件名	<code>server/src/services/bookingService.js</code>
功能概要	客户取消本人预约
参数	<code>customerId: string (l)</code> ； <code>id: string (l</code> ，预约编号)
返回值	<code>booking</code> （更新后的预约单）
详细说明	校验归属、状态； <code>done</code> 抛 <code>ALREADY_DONE</code> ， <code>cancelled</code> 抛 <code>ALREADY_CANCELLED</code> ；合法则置 <code>cancelled</code> 并记录 <code>cancelledAt</code>
使用注意事项	仅 <code>booked</code> 态可取消；需 <code>await repo.flush()</code>

表 5-6 `userService.login()` 接口说明

项	内容
函数名	<code>login</code>
文件名	<code>server/src/services/userService.js</code>
功能概要	登录校验、失败锁定、签发 JWT
参数	<code>phone: string (l)</code> ； <code>password: string (l)</code>
返回值	<code>{token, user}</code>
详细说明	校验手机号格式；查询锁定记录；用户不存在/密码错误记失败；护工离职拒绝；成功清除失败记录并签发 JWT
使用注意事项	达到 <code>LOGIN_MAX_FAILS</code> 锁定 <code>LOGIN_LOCK_MINUTES</code> 分钟；密码比对用 <code>bcrypt</code>

5.3 关键数据结构定义

表 5-7 关键数据结构

结构名	字段	说明
User	<code>id</code> , <code>role</code> , <code>phone</code> , <code>name</code> , <code>age</code> , <code>gender</code> , <code>salary</code> , <code>status</code> , <code>allergy</code> , <code>disease</code> , <code>preference</code> , <code>skills[]</code> , <code>createdAt</code>	用户对象（不含 <code>passwordHash</code> ）
Project	<code>id</code> , <code>name</code> , <code>location</code> , <code>fee</code> , <code>capacity</code> , <code>status</code> , <code>duration</code> , <code>flow</code> , <code>suitable</code> , <code>taboo</code> , <code>createdAt</code>	服务项目

结构名	字段	说明
Booking	id, customerId, workerId, projectId, serviceDate, slot, status, createdAt, updatedAt, cancelledAt	预约记录
LoginLock	phone, failCount, lockedUntil	登录失败锁定
ScheduleCell	serviceDate, slot, booking: Booking null	排班网格单元（空档为 null）
ApiResponse	ok, data?, msg? / ok:false, code, msg	统一响应体

6 核心业务流程设计

6.1 登录与失败锁定

- 校验手机号格式；查询锁定记录，未到期返回 429 ACCOUNT_LOCKED（含剩余分钟）。
- 用户不存在 → 记一次失败 → 401 PHONE_NOT_REGISTERED。
- 护工离职 → 401 WORKER_RESIGNED（不记失败）。
- bcrypt 比对密码：失败 → 记一次失败（达到 LOGIN_MAX_FAILS=5 则锁定 LOGIN_LOCK_MINUTES=15 分钟，计数清零）并返回剩余次数；成功 → 清除失败记录、签发 JWT（有效期默认 8 小时）、返回 {token,user}。

6.2 注册与管理员建号

- 自助注册**（护工/客户）：校验角色白名单、手机号唯一、8 位数字密码、姓名/年龄/性别；护工可带技能（须为真实项目编号）、客户可带健康档案；bcrypt 哈希后入库；在 await 哈希前后**双检**手机号唯一，防并发重复注册。
- 管理员建号**：密码缺省 12345678；字段按角色白名单过滤。

6.3 预约流程（核心）

bookingService.book() 纯同步完成以下校验链，任一失败即拒绝且不产生半途写入：

1. rollover()

2. assertDateInWindow()

3. assertProjectBookable()

4. 客户时段去重

5. 项目容量校验

6. eligibleWorkers()

7. 随机分配护工

8. 生成 booked 记录并 save()

先过账，保证“过期即完成”

日期格式合法且 ∈ [今天, 今天+窗口天数)

项目存在且 status=active

同一客户同日同时段至多 1 条 booked（跨项目）

该 项目+日期+时段 的 booked 数 < capacity

护工池 = 在职 active n 技能含该项目 n 该时段未被占用

为空 → 409 NO_WORKER

从候选池随机取一名

6.4 取消预约

仅本人可取消；done 拒绝 (ALREADY_DONE)、cancelled 拒绝 (ALREADY_CANCELLED)；合法则置为 cancelled 并记录 cancelledAt。

6.5 过账 Rollover

服务日期早于今天的 booked 记录统一置为 done。该操作幂等，在服务启动时以及每次涉及预约的读/写（预约、取消、窗口/历史查询、看板统计）前调用，保证「过期即完成」的一致性。

6.6 看板统计

- **管理员**：客户数、护工数/在职数、项目数/进行中数、预约总数、今日已约、窗口内已约，以及窗口内近期待服务预约明细（最多 100 条，含客户/项目/护工名）。
- **客户**：进行中项目数、窗口内我的预约（myUpcoming）、我的已完成（myDone）、我的总预约（myTotal）。
- 前端在此真实统计之上补充可视化（近 7 日趋势、时段分布、项目热度、护工服务量），所有图表锚定真实计数与真实项目/护工名称。

6.7 护工排班与隐私隔离

视角	接口	每格内容
护工本人	/api/schedule/me	预约详情 含 客户健康档案 （过敏/疾病/偏好）
管理员	/api/users/:id/schedule	仅客户编号与姓名， 不含健康档案 （按隐私约定从服务层裁剪字段）

排班/窗口视图以「日期 × 4 时段」网格返回，无预约的格返回 `booking: null`（空档）。

7 出错处理设计

层次	机制
入参校验	<code>utils/validate.js</code> 统一校验并抛 <code>HttpError(400, code, msg)</code> ，逐字段明确错误码
业务规则	服务层抛语义化 <code>HttpError</code> （如 409 <code>SLOT_TAKEN</code> ），不产生部分写入
异步异常	路由以 <code>ah()</code> 包装，Promise 拒绝进入统一错误处理器
统一出口	<code>errorHandler</code> 将 <code>HttpError</code> 转为 <code>{ok:false,code,msg}</code> ；非预期异常记日志并返回 <code>500 INTERNAL</code> （不泄露堆栈）
前端	请求层将失败归一为 <code>ApiError</code> ；401 <code>UNAUTHORIZED</code> （非登录注册）自动清会话并跳转登录页；业务错误由各视图 <code>ElMessage</code> 提示
存储	MySQL 落库失败记日志、内存为准、下次写自愈；连接断开自动重连；JSON 文件损坏自动备份重建
启动	<code>index.js</code> 捕获 boot 异常，打印「请检查 STORE / DB_* 配置与 MySQL 服务」并退出

8 安全与保密设计

方面	措施
密码存储	<code>bcryptjs</code> 加盐哈希（ <code>BCRYPT_ROUNDS=10</code> ），服务端不落明文、接口不返回 <code>passwordHash</code>
身份认证	JWT 签名（HS256），有效期默认 8 小时；密钥经 <code>JWT_SECRET</code> 注入
权限控制	<code>authRequired</code> （认证）+ <code>requireRole</code> （角色守卫）双层；前端路由守卫按 <code>role</code> 拦截越权访问
暴力破解	连续失败 5 次锁定 15 分钟（可配）
隐私隔离	客户健康档案仅本人与服务护工可见；管理员排班视图经服务层裁剪
输入约束	手机号 11 位、密码 8 位数字、年龄 0-150、时段 0-3、日期 <code>YYYY-MM-DD</code> ，服务端全部复校

方面	措施
传输与部署	开发环境经同源 <code>/api</code> 代理；生产建议启用 HTTPS；CORS 由 <code>cors()</code> 控制
敏感文件	<code>.env</code> （含数据库口令、JWT 密钥）永不入库，仅提交 <code>.env.example</code> ；运行数据、日志、依赖目录经 <code>.gitignore</code> 排除
数据库账号	应用使用仅授 DML 的最小权限账号；建表/迁移由 root 手工执行

9 界面（UI）设计概要

- **总体风格**：米白背景（`#f7f2ea`）+ 墨绿主色（`#2b6349` / `#2d6a4f`）+ 金色点缀（`#c9a24b`）；适老化 16px 基准字号、高对比、非彩虹配色。
- **统一布局**：顶栏标题 + 园区地图/环境看板 + 角色导航（`RoleLayout`），会话 30 分钟无操作自动退出。
- **角色界面**：
 - 管理员：数据看板（统计卡 + 图表行 + 近期待服务预约表）、客户管理、护工管理、项目管理。
 - 护工：个人中心（只读）、服务时间状态（3×4 网格）、项目总览。
 - 客户：个人中心、数据看板、我的项目（3×4 网格 + 历史）、项目总览与预约、AI 咨询。
- **图表规范**：`BaseChart` 封装 ECharts，按需注册折线/柱状/饼/雷达；图表在数据就绪后以 `v-if` 挂载，高度经 `px()` 归一化，`ResizeObserver` 自适应且跳过 0 宽容器。
- **反馈**：操作结果以 Element Plus 消息/对话框提示，空态使用 `el-empty`，加载态使用 `v-loading`。

10 可维护性与可扩展性设计

1. **分层清晰、依赖单向**：路由 → 服务 → 存储，服务层不感知 HTTP，存储层不感知业务。
2. **存储可插拔**：新增存储实现只需满足 `repo` 约定并替换入口装配点，业务与前端零改动。
3. **规则集中**：预约与容量规则集中在 `bookingService`；字段校验集中在 `utils/validate.js`；日期语义集中在 `utils/datetime.js`。
4. **配置外置**：窗口天数、锁定阈值、端口、密钥、数据库连接均可经 `.env` 调整。
5. **可测试**：服务层为纯同步函数、支持注入 `now`；`npm run selfcheck` 提供 72 项业务断言（基于文件引擎，不改动正式数据）。
6. **前端可扩展**：请求层与接口封装分离，新增接口只需在 `api/index.js` 增一行、视图复用现成组件。
7. **数据可交付**：建表/灌数脚本入库，`db:seed` 可按当天重新生成演示数据，便于换机与交付。

11 运行环境与部署设计

11.1 运行环境

项目	要求
操作系统	Windows / macOS / Linux（开发环境为 Windows 11）
运行时	Node.js 18+（ESM），npm
数据库	MySQL 8.x，字符集 <code>utf8mb4</code> ，库名 <code>forest_oxygen</code>
浏览器	Chrome / Edge 等现代浏览器
端口	前端 3000，后端 3001，MySQL 3306

11.2 部署结构

开发/演示环境为单机双进程：前端 Vite 开发服务器 (:3000) 通过 `/api` 代理转发到后端 Express (:3001)，后端连接本机 MySQL。生产构建时前端产物为静态资源 (`dist/`，`npm run build`)，可由任意静态服务器托管，并将 `/api` 反向代理到后端。



11.3 初始化与数据库交付

- 1. 建库建表：`mysql -u root -p < server/sql/schema-mysql.sql` (可重复执行)。
- 2. 灌入演示数据：`mysql -u root -p < server/sql/seed.sql` (脚本内含 `USE forest_oxygen`)；如需围绕「当天」刷新窗口预约，先执行 `cd server && npm run db:seed` 重新生成。
- 3. 配置：由 `server/.env.example` 复制为 `server/.env`，设置 `STORE=mysql` 及 `DB_*`。**注意：**`STORE` 代码默认值为 `file`，未创建 `.env` 或未设 `STORE=mysql` 时会回退到 `JSON` 文件存储。
- 4. 启动：后端 `cd server && npm install && npm start`；前端 `npm install && npm run dev`。
- 5. 空库自动引导：首次启动若库中无管理员，自动创建配置的引导管理员 (`ADMIN_PHONE`) 与 8 个项目目录。

数据库不在版本库中，仓库仅提交「建表脚本 + 灌数脚本 + 配置模板」，他人在本机导入 SQL 并配置 `.env` 即可获得完整可运行环境。

12 附录

12.1 固定时段 (`src/utils/config.js`)

slot	名称	时间
0	上午①	08:30–10:00
1	上午②	10:30–12:00
2	下午①	14:00–15:30
3	下午②	16:00–17:30

12.2 环境数据合理区间 (园区环境看板)

指标	区间
园区温度	18–28 °C
空气湿度	45–85 %RH
PM2.5	5–35 µg/m³
负氧离子	2000–6500 个/cm³

12.3 服务项目目录 (默认 8 项，`server/src/store/catalog.js`)

编号	名称	地点	费用(元)	容量
P001	森林浴·负氧离子漫步	湖畔步道	88	8
P002	太极养生操	晨练草坪	68	10
P003	中医艾灸理疗	养生馆·二楼灸疗室	168	4
P004	中药足浴	足浴SPA区	98	6
P005	药膳养生餐	氧吧·食疗餐厅	128	20
P006	森林冥想·正念练习	竹林冥想亭	58	12
P007	八段锦晨练	松林广场	48	15
P008	园艺疗法·花艺种植	四季花圃	88	6

12.4 文件修改控制

修改编号	版本	修改条款及内容	修改日期
1	V1.0	创建	2026-09-01
2	V1.1	补充预约窗口、过账、隐私隔离设计	2026-09-11
3	V1.2	按概要设计模板重构章节结构，补充模块结构图、类图、接口函数详细信息、关键数据结构	2026-09-15